

Week 1 – Building Noah's LLM Core

Objective

The goal of Week 1 is to establish the core Large Language Model (LLM) infrastructure that will power Noah. At this stage, the backend should be capable of receiving a natural language request from the Flutter application, forwarding it to a language model, and returning a structured JSON response.

No application functionality, Retrieval-Augmented Generation (RAG), recommendation systems, or database integration will be implemented during this phase.

Learning Objectives

During this week, developers should become familiar with:

- FastAPI
 - REST APIs
 - Local and cloud-hosted LLMs
 - Prompt Engineering
 - JSON-based structured outputs
 - Basic Intent Recognition
 - Python project organisation
-

Folder Structure

Create the following project structure.

```
noah-backend/
```

```
|
```

```
├─ app/
```

```
|   └─ api/
```

```
|   └─ llm/
|
|   └─ planner/
|
|   └─ models/
|
|   └─ core/
|
|   └─ main.py
|
|
└─ requirements.txt
└─ .env
└─ README.md
```

Only these folders are required during Week 1.

Required Python Packages

Install the following packages.

```
pip install fastapi
pip install uvicorn
pip install pydantic
pip install python-dotenv
pip install httpx
pip install loguru
```

For cloud-based models (recommended during development):

Choose one provider:

- Groq
- OpenRouter
- Gemini

Later these can be replaced by Ollama.

Backend Components

The following files should be created.

`main.py`

Responsibilities

- Initialise FastAPI
- Register routes
- Configure middleware
- Start the backend

`api/chat.py`

Responsibilities

- Receive user messages
- Validate requests
- Forward requests to the planner
- Return JSON responses

`api/health.py`

Responsibilities

- Verify backend availability
- Return server status

Example

```
{  
    "status": "running"  
}
```

`planner/planner.py`

Responsibilities

The planner acts as the decision layer between the API and the language model.

Initially its responsibility is simple:

User Request

↓

Forward Request

↓

LLM

Future versions will perform tool selection and planning.

llm/llm_service.py

Responsibilities

- Connect to the selected LLM provider
- Send prompts
- Receive responses
- Handle API failures
- Return structured outputs

The LLM implementation should be isolated from the rest of the application so that changing providers requires minimal code modifications.

models/chat_models.py

Create request and response schemas.

Example

```
ChatRequest
```

```
ChatResponse
```

These models define the communication protocol between Flutter and the backend.

Prompt Engineering

The language model should not generate arbitrary conversational text.

Instead, it should always return structured JSON.

Example system prompt

You are Noah.

Your task is to analyse the user's request.

Determine:

1. Intent
2. Entities
3. Required tool
4. Response type

Return ONLY valid JSON.

Expected Output

Input

Show me my Payback barcode

Expected Output

```
{
  "intent": "OPEN_BARCODE",
  "tool": "barcode_tool",
  "entities": {},
  "response_type": "functional"
}
```

Input

Find frozen pizzas below €2

Expected Output

```
{
  "intent": "SEARCH_PRODUCT",
  "tool": "merchant_search",
```

```
"entities":
{
    "product": "Frozen Pizza",
    "max_price": 2
},
"response_type": "hybrid"
}
```

The language model is therefore acting as an intelligent parser rather than a conversational chatbot.

Dataset Construction

Rather than immediately training a model, begin constructing a supervised dataset for future fine-tuning.

Each example should contain:

```
{
    "instruction": "...",
    "intent": "...",
    "tool": "...",
    "entities": {},
    "response_type": "..."
}
```

Initially, manually create examples for approximately 20–30 core intents.

Each intent should contain 100–200 natural language variations.

The resulting dataset will later serve as the foundation for supervised fine-tuning of a smaller local model.

Artificial Dataset Generation

Manually creating thousands of training examples is time-consuming and often results in limited linguistic diversity. Instead, Noah's intent dataset will be bootstrapped using Large Language Models to automatically generate semantically equivalent paraphrases for each canonical instruction.

Objective

Expand a small set of manually verified examples into a large supervised dataset suitable for intent recognition and future fine-tuning.

Process

1. Define a canonical user instruction.

Example

```
Show me my Payback barcode.
```

2. Assign the corresponding structured output.

```
{  
  "intent": "OPEN_BARCODE",  
  "tool": "barcode_tool",  
  "entities": {},  
  "response_type": "functional"  
}
```

3. Use an LLM to generate 100–200 diverse paraphrases while preserving the original intent.

Example generated paraphrases

```
Show my barcode.
```

```
Open my loyalty card.
```

```
Display my Payback QR code.
```

```
I need my barcode.
```

```
Where is my Payback card?
```

```
Can you open my barcode?
```

```
Bring up my digital loyalty card.
```

Display my membership barcode.

Show my rewards card.

Open my Payback barcode.

4. Associate every generated paraphrase with the same structured JSON label.
5. Automatically validate the generated dataset by checking JSON syntax, duplicate entries, and intent consistency.
6. Perform manual quality assurance on a random subset (e.g., 5–10%) to verify that generated paraphrases preserve the intended meaning.

Advantages

- Significantly reduces manual annotation effort.
- Produces a linguistically diverse dataset.
- Improves model robustness to different user phrasings.
- Creates a scalable pipeline for adding new intents.
- Enables rapid expansion from a few dozen examples to several thousand training instances.

Target Dataset Size

Component	Target
Core Intents	20–30
Canonical Examples per Intent	1–5
Generated Paraphrases per Intent	100–200
Total Training Examples	5,000–10,000

The resulting dataset will serve as the primary supervised training corpus for Noah's intent prediction model and provide high-quality examples for prompt engineering and future supervised fine-tuning (SFT).

Week 1 Deliverables

At the end of Week 1, the following should be functional.

- FastAPI backend running
- `/health` endpoint operational
- `/chat` endpoint operational
- LLM connected through a cloud API
- Structured JSON responses
- Initial prompt template
- Initial intent dataset
- Basic project documentation

No application actions, memory, tool execution, recommendation system, or RAG components should be implemented during this phase.